

Docs

<https://github.com/ryukzak/wrench/tree/master/docs>

acc32

reverse_string_pstr

```
def reverse_string_pstr(s: str) -> tuple[str | list[int], str]:
    """Reverse a Pascal string.

    - Result string should be represented as a correct Pascal string.
    - Buffer size for the message -- `0x20`, starts from `0x00`.
    - End of input -- new line.
    - Initial buffer values -- `_`.

    Python example args:
        s (str): The string with `\n` as end of the input.

    Returns:
        tuple: A tuple containing the reversed string and an empty string.
    """
    line, rest = read_line(s, 0x20)
    if line is None:
        return [overflow_error_value], rest
    return line[::-1], rest

assert reverse_string_pstr('hello\n') == ('olleh', '')
# and mem[0..31]: 05 6f 6c 6c 65 68 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f
assert reverse_string_pstr('world!\n') == ('!dlrow', '')
# and mem[0..31]: 06 21 64 6c 72 6f 77 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f 5f
```

f32a

count_ones

```
def count_ones(n: int) -> int:
    """Count the number of ones in the binary representation of a number"""
    count = 0
    while n > 0:
        count += n & 1
        n >>= 1
```

```

    return count

assert count_ones(5) == 2
assert count_ones(7) == 3
assert count_ones(247923789) == 13
assert count_ones(2147483647) == 31

```

m68k

glob_match

```

def glob_match(input: str) -> tuple[list[int], str]:
    """Match a text against a glob pattern.

    Input format:
        <pattern>\n
        <text>\n

    - `?` matches exactly one character.
    - `*` matches any sequence of characters, including an empty one.
    - Any other character matches only itself.
    - Buffer size for every line -- `0x20`, starts from `0x00`.
    - Returns 1 when the text matches the pattern and 0 otherwise.
    - A recursive solution with backtracking is recommended.

    Python example args:
        input (str): The input string with two lines.

    Returns:
        tuple: A tuple containing the match result and the remaining input.
    """
    pattern, rest = read_line(input, 0x20)

    if pattern is None:
        return [overflow_error_value], rest

    text, rest = read_line(rest, 0x20)

    if text is None:
        return [overflow_error_value], rest

    def match(p: str, t: str) -> bool:
        if p == "":
            return t == ""

        if p[0] == "*":
            return match(p[1:], t) or (t != "" and match(p, t[1:]))

        if t == "":

```

```

        return False

    if p[0] == "?" or p[0] == t[0]:
        return match(p[1:], t[1:])

    return False

return [1 if match(pattern, text) else 0], rest

assert glob_match('a*c\nabc\n') == ([1], '')
assert glob_match('a?c\nabc\n') == ([1], '')
assert glob_match('a?c\nabbc\n') == ([0], '')
assert glob_match('*.txt\nfile.txt\n') == ([1], '')

```

risc-iv

sum_odd_n

```

def sum_odd_n(n: int) -> int:
    """Calculate the sum of odd numbers from 1 to n"""
    if n <= 0:
        return -1
    total = 0
    for i in range(1, n + 1):
        if i % 2 != 0:
            total += i
    return total

assert sum_odd_n(5) == 9
assert sum_odd_n(10) == 25
assert sum_odd_n(90000) == 2025000000

```

scheme

f32a-neumann[-microcode]